

Labwork 8: Scatter

Pham Tien Nhat

October 4, 2026

1 Implementation

The input image of this labwork is stored as an Array of Struct (AoS) where each pixel keeps its R, G and B values together. In Labwork 8 I have converted the input image to HSV as a Struct of Array (SoA), with 3 different arrays $H[]$, $S[]$ and $V[]$, then I have also converted the HSV image version back to RGB original image.

```
import math
import time
import numpy as np
import matplotlib.pyplot as plt

image = plt.imread('1.jpeg')
plt.imshow(image)
h, w, d = image.shape
```



Figure 1: Input image.

RGB to HSV Conversion: Each thread reads one pixel, scales R, G, B from $[0..255]$ to $[0..1]$, and finds max , min and $\Delta = max - min$ following the formula given in the slide. The hue $H \in [0, 360]$ depends on which channel is the maximum, the saturation is $S = \Delta / max$ (0 if $max = 0$) and the value is $V = max$. The kernel is a scatter: one input pixel (R, G, B) is written to 3 different output arrays.

```
@cuda.jit
def RGB2HSV(src, H, S, V):
    tid = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    tidy = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if tid >= src.shape[1] or tidy >= src.shape[0]:
        return
```

```

r = src[tidy, tidy, 0] / 255
g = src[tidy, tidy, 1] / 255
b = src[tidy, tidy, 2] / 255
mx = max(r, g, b)
mn = min(r, g, b)
delta = mx - mn

# H conversion
if delta == 0:
    hue = 0.0
elif mx == r:
    hue = 60 * ((g - b) / delta) % 6
elif mx == g:
    hue = 60 * ((b - r) / delta + 2)
else:
    hue = 60 * ((r - g) / delta + 4)

# S conversion
if mx == 0:
    sat = 0.0
else:
    sat = delta / mx

# scatter: one pixel (R, G, B) goes to 3 different arrays
H[tidy, tidy] = hue
S[tidy, tidy] = sat
V[tidy, tidy] = mx

blockSize = (32, 32)
gridSize = (math.ceil(w / blockSize[0]), math.ceil(h / blockSize[1]))
devSrc = cuda.to_device(image)
devH = cuda.device_array((h, w), np.float32)
devS = cuda.device_array((h, w), np.float32)
devV = cuda.device_array((h, w), np.float32)
RGB2HSV[gridSize, blockSize](devSrc, devH, devS, devV)

```

The output of HSV process is display in the figure below, with three images each represents a channel from HSV, where H represents the color changes, value is between 0 to 360, S shows the colorfulness, value lies within 0 and 1 and V displays the brightness, value ranging from 0 to 1

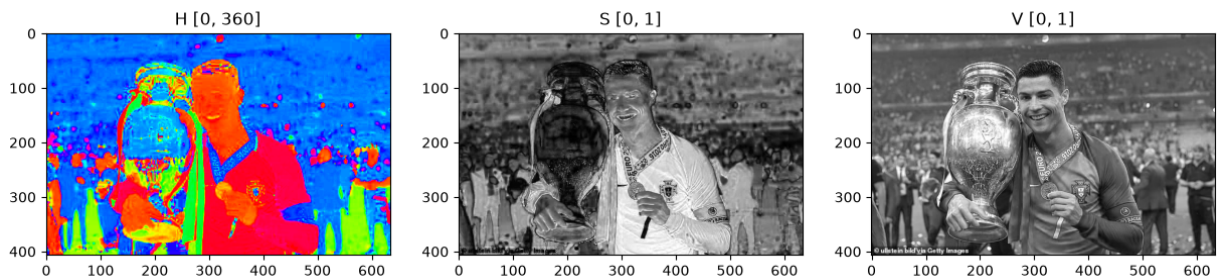


Figure 2: The 3 output arrays H , S and V .

HSV to RGB Conversion: The kernel reads the 3 arrays H , S , V and writes one RGB image.

```

@cuda.jit
def HSV2RGB(src, dst, H, S, V):
    tidy = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    tidy = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if tidy >= src.shape[1] or tidy >= src.shape[0]:
        return

```

```

# Preparation
d = H[tidy, tidy] / 60
hi = int(d) % 6
f = d - hi
l = V[tidy, tidy] * (1 - S[tidy, tidy])
m = V[tidy, tidy] * (1 - f * S[tidy, tidy])
n = V[tidy, tidy] * (1 - (1 - f) * S[tidy, tidy])

# Conversion
if 0 <= H[tidy, tidy] < 60:
    R = V[tidy, tidy]
    G = n
    B = l
elif 60 <= H[tidy, tidy] < 120:
    R = m
    G = V[tidy, tidy]
    B = l
elif 120 <= H[tidy, tidy] < 180:
    R = l
    G = V[tidy, tidy]
    B = n
elif 180 <= H[tidy, tidy] < 240:
    R = l
    G = m
    B = V[tidy, tidy]
elif 240 <= H[tidy, tidy] < 300:
    R = n
    G = l
    B = V[tidy, tidy]
else:
    R = V[tidy, tidy]
    G = l
    B = m
R = R * 255
G = G * 255
B = B * 255

dst[tidy, tidy, 0] = R
dst[tidy, tidy, 1] = G
dst[tidy, tidy, 2] = B

devDst = cuda.device_array((h, w, 3), np.uint8)
HSV2RGB[gridSize, blockSize](devSrc, devDst, devH, devS, devV)
result = devDst.copy_to_host()

```

2 Experimental Results

Figure ?? compares the input image with the output after RGB to HSV and RGB conversions.

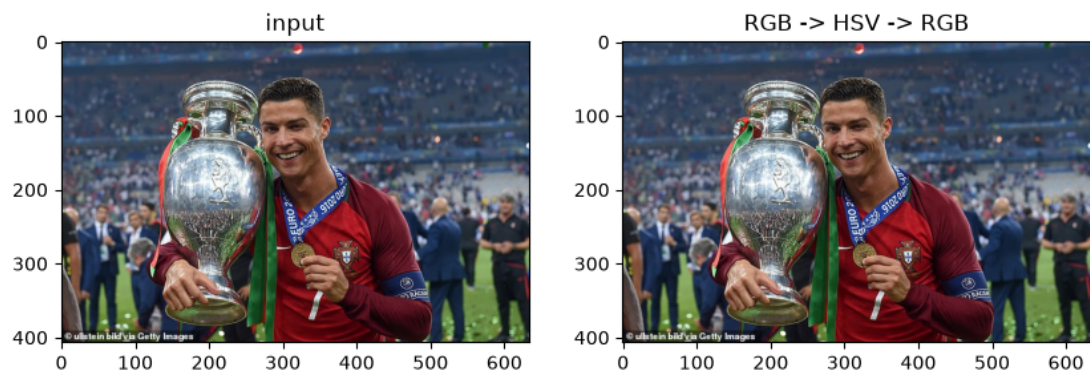


Figure 3: Input image (left) and output after $\text{RGB} \rightarrow \text{HSV} \rightarrow \text{RGB}$ (right).

The two images look similar to each other after the conversions step.